



12

Hosting a Blazor Application in WinUI

Blazor is a web framework from Microsoft that allows .NET developers to create C# web applications with little to no JavaScript code. Server-side Blazor applications were introduced with ASP.NET Core 3.0, and ASP.NET Core 3.2 added the ability to create client-side Blazor web apps with **WebAssembly (Wasm)**. Wasm (<https://webassembly.org/>) allows runtimes such as .NET and Java to run in web applications in the browser, and it is supported by all modern browser engines. By leveraging the **WebView2** control in WinUI 3, Windows developers can run a cloud-hosted Blazor application inside their WinUI client application. These options are changing a bit in .NET 8 with the introduction of **rendering modes**. We'll discuss these new modes and the advantages of each.

In this chapter, we will cover the following topics:

- Learning some basics of client-side .NET development with ASP.NET Core and Blazor
- Creating a new Blazor application with **Visual Studio Code** and the **.NET command-line interface (CLI)**
- Deploying Blazor applications to the **Azure Static Web Apps** service
- Creating a WinUI application to host a Blazor application in a **WebView2** browser control

By the end of this chapter, you will understand how to create a new Blazor application, deploy it to the cloud, and use the application as a **single-page application (SPA)** inside the WinUI **WebView2** control.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1809 (build 17763) or later or Windows 11.
- Visual Studio 2022 or later with the .NET Desktop Development workload configured for Windows App SDK development.
- **Visual Studio Code (VS Code)** with the following extensions: C# Dev Kit and Debugger for Microsoft Edge.
- Windows Terminal (which was built with WinUI) or your preferred command-line tool. You can also use the **Terminal** window in VS Code.
- To create the Blazor project, install .NET 7 SDK or later.

The source code for this chapter is available on GitHub at this URL:

<https://github.com/PacktPublishing/-Learn-WinUI-3/tree/master/Chapter12>.

Getting started with ASP.NET Core and Blazor

Blazor is a web development framework that provides C# developers with an alternative to JavaScript when building client-side web applications. Blazor is a part of **ASP.NET Core** and was first introduced with ASP.NET Core 3.0. Let's start by exploring a brief history of **ASP.NET** and ASP.NET Core.

A brief history of ASP.NET and ASP.NET Core

ASP.NET was Microsoft's .NET-based web development framework that was first released in 2002. The early versions of ASP.NET used a client development model called **Web Forms**, which was intended as a web equivalent of **Windows Forms (WinForms)** client applications. Web Forms was popular with .NET web developers but did not adhere to many web development best practices and patterns. Many developers were critical of the large amount of **ViewState** data sent over the wire with every server request and response.

In response to Web Forms criticism, the ASP.NET team released ASP.NET MVC in 2009. Web applications built with ASP.NET MVC follow the **Model-View-Controller (MVC)** pattern. The new framework was well-received by the .NET community and is still a popular choice with web developers today. ASP.NET was also one of the first Microsoft frameworks to be released as open source. In 2012, ASP.NET MVC 4 was released as open source under the Apache License 2.0.

As the .NET team continued to embrace open source software, they also decided to make a fresh start with a new, open source, and cross-platform version of .NET called **.NET Core**. Microsoft released .NET Core 1.0 in 2016 with runtimes available for Windows, macOS, and Linux. With the release of .NET Core came a new web framework called **ASP.NET Core**. ASP.NET Core 1.0 included project templates to build web applications and Web API projects. The web applications were built with the MVC pattern, the **Razor** syntax for building a rich UI, and CSS for styling pages.

The ASP.NET team continued to add more features to ASP.NET Core over the next several years, including the following:

- **Razor Pages:** Razor Pages projects were introduced with ASP.NET Core and offer a simple alternative to ASP.NET Core MVC

- **SignalR:** A framework for real-time web communication; SignalR is integral to client-server communication in Blazor server applications
- **Identity (previously Identity Core):** Supports login functionality in ASP.NET Core applications and manages authentication resources such as users, passwords, roles, tokens, and so on

NOTE

This book will not provide a detailed tutorial on ASP.NET Core development. If you want to learn more about building web applications with ASP.NET Core, see ASP.NET Core 5 for Beginners, by Andreas Helland, Vincent Maverick Durano, Jeffrey Chilberto, and Ed Price, Packt Publishing (<https://www.packtpub.com/product/asp-net-core-5-for-beginners/9781800567184>).

But where does Blazor fit into the ASP.NET Core development picture? Let's explore that next.

What is Blazor?

Blazor is a framework for building web applications with .NET and C#. Prior to .NET 8, there were three hosting models from which developers could choose when starting a new Blazor project:

- **Blazor Server:** Introduced with ASP.NET Core 3.0, the server model executes application logic on the server with UI updates pushed to the client through SignalR connections.
- **Blazor WebAssembly:** Delivered later with ASP.NET Core 3.2, this execution model runs solely on the client, sandboxed and running on the browser's UI thread via Wasm.
- **Blazor Hybrid:** This is a newer type of Blazor app that combines web and native client technologies. In this model, Razor components run in .NET on the client and render web user interfaces to the native client with interop and WebView technology that is

seamless to the user. Hybrid apps can be used with .NET MAUI, WPF, and WinForms clients.

In .NET 8 and later, the Blazor team has evolved these hosting models into rendering modes. There are now three modes you can choose when developing with Blazor:

- **Server mode:** The equivalent of the server hosting model from previous versions.
- **WebAssembly mode:** The client-side mode that leverages Wasm.
- **Auto mode:** The new Auto mode combines the best of the previous two modes. It will run in WebAssembly mode if the client runtime components can be downloaded quickly enough. Otherwise, it falls back to Server mode.

So, which mode should you choose for your next Blazor application? Luckily, with .NET 8, you don't have to choose the same mode for the entire application anymore. The modes can be selected at the component level by setting the `@rendermode` attribute. That component-level decision will depend on your project's requirements, but it's likely that many applications will start to leverage Auto mode.

NOTE

At the time of writing, .NET 8 is still only available as developer pre-views. We will build the application in this chapter with .NET 7 and the Blazor WebAssembly hosting model. You will be able to build the same kind of app with .NET 8 and the WebAssembly mode.

Here are some of the pros and cons of the server and Wasm hosting models:

Blazor Hosting Models

	Blazor Server	Blazor WebAssembly
Pros	• Smaller initial download • Full .NET API available • App code stays on server • Better performance on resource-limited devices • Robust debugging	• No .NET needed on server • Serverless deployments • Workload distributed to clients; increased scalability • Leverages client capabilities • Offline execution as a Progressive Web App (PWA)
Cons	• No offline support • Higher latency • Clients rely on server connections; less scalability • ASP.NET Core server hosting required • No serverless	• Limited by browser capabilities • Runtime included in initial download; slower first load • .NET runtime limited on Wasm compared for full .NET API set • Client-side debugging complicated in production

Figure 12.1 – Blazor hosting model pros and cons

The Blazor server hosting model was released first and has the most mature tooling and debugging support. It is a great choice if you plan to host the server on a service that supports ASP.NET Core and your users may be using browsers that do not have Wasm support.

In this chapter, we are going to focus on client-side Blazor applications. So, why choose this model, and how does it work?

WebAssembly and client-side .NET development

The primary benefits of the client-side hosting model with Wasm are the option of serverless deployment and the ability for clients to work offline. The offline support means your Blazor application can be configured as a **Progressive Web Application (PWA)** and downloaded to PCs, tablets, and phones. You can learn more about PWAs from Mozilla's developer documentation:

https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps.

The reasons that Blazor client applications can run as PWAs are the reasons why we want to use it in a **WebView2** control in our WinUI application. Once the web application has loaded in the browser host, all

in-memory execution and interactions can occur regardless of any interruptions in network connectivity. If connectivity, scalability, and server hosting are not concerns in your project, then the Blazor server model could certainly be used.

Simple Blazor Wasm applications can be hosted as *static resources* on a web server. You can also host Blazor Wasm applications on ASP.NET Core web hosting solutions. Doing this enables sharing code with other web solutions on the server and more advanced routing scenarios to support deep linking within the application. With .NET 8, using a host with ASP.NET Core support allows you to take advantage of the new Auto mode or choose to only use Wasm mode in some components that lend themselves to offline use.

With the Wasm model, when clients make their first request to the server, the entire application and .NET runtime are sent to the browser in the response, and the entire application runs on the client side. There is no shared server-side code in this mode. The runtime and application are then loaded on top of Wasm on the UI thread:

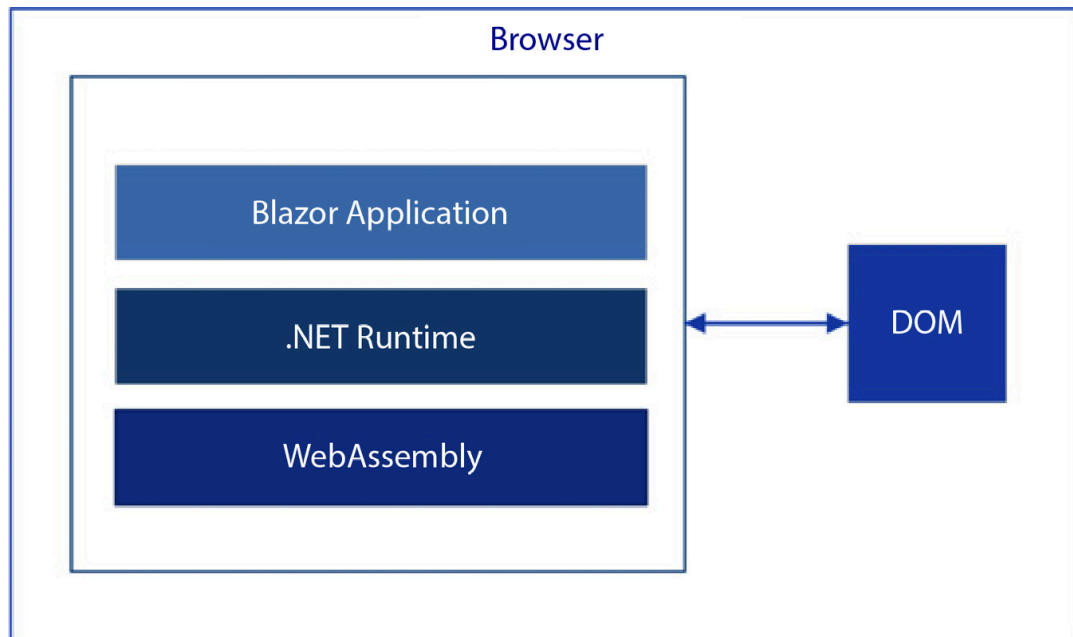


Figure 12.2 – The Blazor Wasm model running in the browser

Now that you have a little background on ASP.NET Core and Blazor applications, let's create a Blazor Wasm project and get some hands-on experience with the framework.

Creating a Blazor Wasm application

It's time to start building the Blazor application that we'll be running in our WinUI application. We are going to use the .NET CLI and VS Code to create the Blazor project. You can also use Visual Studio 2022 if you prefer the experience of a full-featured IDE:

1. Start by opening a Command Prompt with your Terminal application of choice. I will be using Windows Terminal (<https://apps.microsoft.com/store/detail/windows-terminal/9N0DX20HK701>) with PowerShell 7.3 (<https://learn.microsoft.com/powershell/scripting/overview>):

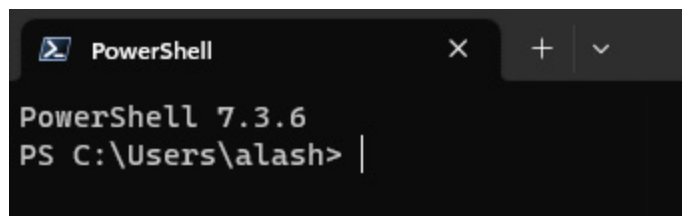


Figure 12.3 – Running PowerShell 7.3 in Windows Terminal

2. Use the terminal to change the current folder to the location where you keep your projects. My location will be **C:\Users\alash\source\repos**.
3. Use the following command to create a new Blazor WebAssembly project named **BlazorTasks** and hit *Enter*: **dotnet new blazorwasm -o BlazorTasks**. The .NET CLI will create the new project, and you should see a message indicating it has completed successfully:


```
The template "Blazor WebAssembly App" was created successfully.
This template contains technologies from parties other than Microsoft, see https://aka.ms/aspnetcore/7.0-third-party-notices for details.

Processing post-creation actions...
Restoring C:\Users\alash\source\repos\BlazorTasks\BlazorTasks.csproj:
  Determining projects to restore...
  Restored C:\Users\alash\source\repos\BlazorTasks\BlazorTasks.csproj (in 6.61 sec).
Restore succeeded.
```

Figure 12.4 – The .NET CLI successfully creates a Blazor WebAssembly App project

4. Navigate to the new **BlazorTasks** folder that .NET just created. If you have VS Code (<https://code.visualstudio.com/>) installed, you can enter **code .** at the command line to open the current folder in VS Code:

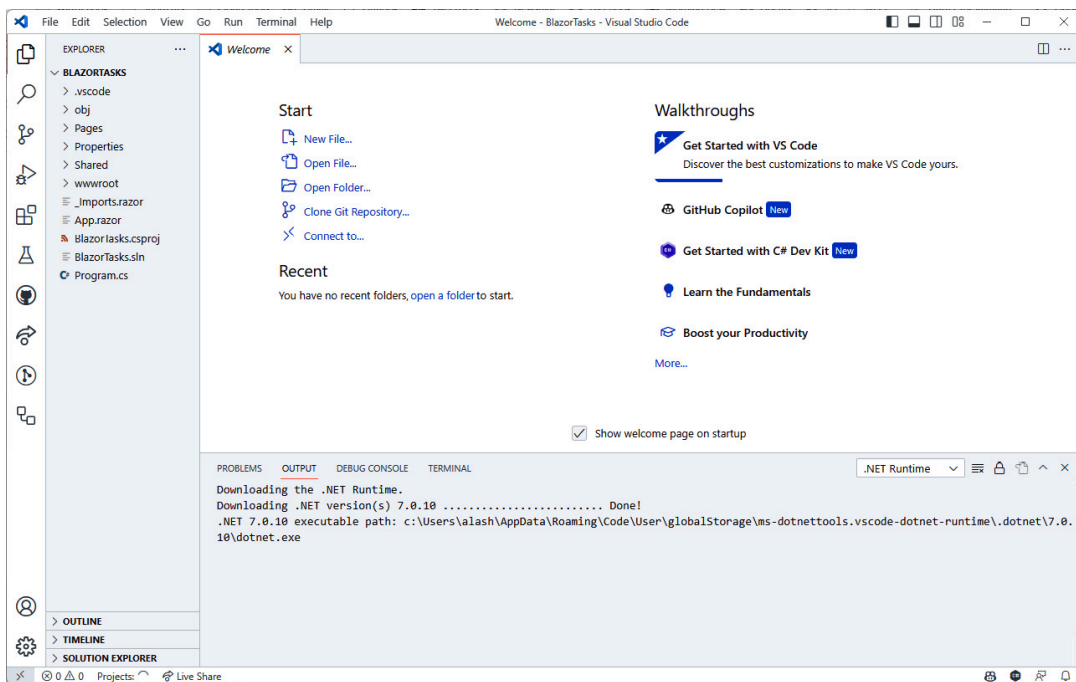


Figure 12.5 – The BlazorTasks project in VS Code

You may see some activity in the **OUTPUT** window as VS Code downloads some debugging and editing tools relevant to the project. If you are presented with a **Do you trust the authors of the files in this folder?** dialog, select the **Yes, I trust the authors** button to continue.

5. Switch to the **Terminal** window in VS Code. If the window isn't visible at the bottom of your editor, you can click **Terminal | New Terminal** from the menu.

6. Type **dotnet run** in the Terminal. You can also use *F5* to run in VS Code as you are accustomed to using in Visual Studio. When the compilation completes, you can view the running **BlazorTasks** application by navigating to **https://localhost:5240/** in your browser (the port number will be displayed in the **TERMINAL** window):

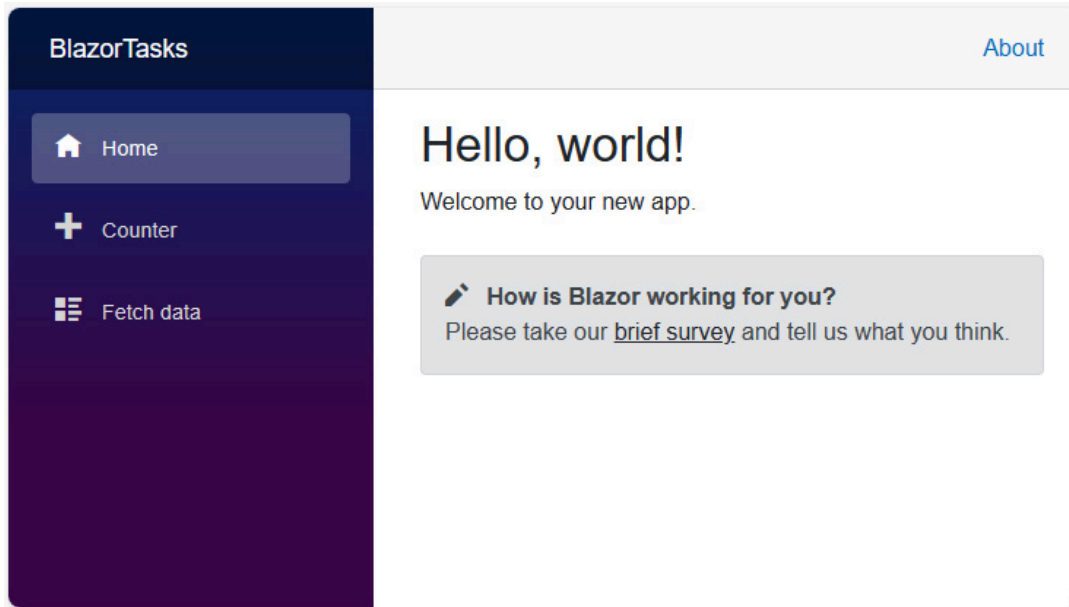


Figure 12.6 – Running the BlazorTasks project for the first time

The default project template has three navigation options in the left pane: **Home**, **Counter**, and **Fetch data**. As you navigate from page to page, all the execution logic is running within the browser. There are no round trips to an ASP.NET Core server instance.

7. You can open the developer tools in your browser by pressing *F12*. You will see that there is no activity on the **Network** tab of the developer tools while navigating to the **Counter** tab in the application and clicking the **Click me** button several times:

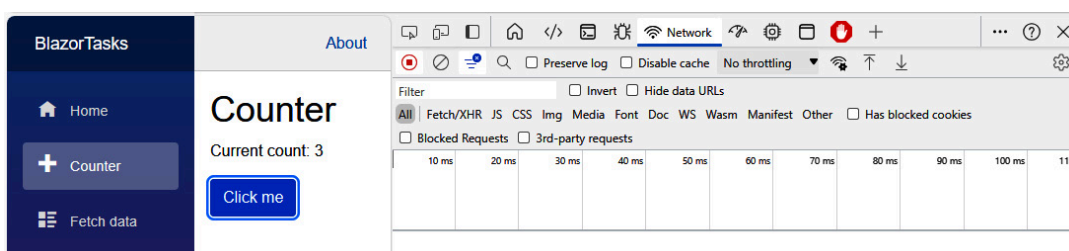


Figure 12.7 – Viewing network activity in the BlazorTasks application

8. Finally, when you are done exploring the application, you can close your browser and hit *Ctrl + C* in the VS Code **TERMINAL** window to stop debugging the application.

Now that we have created and tested the project, let's start coding a new task page for the application.

Building a simple application for tracking tasks

In this section, we are going to create a new task page for the application that will appear in the left navigation below the **Fetch data** item. If you like, you can remove the other components from the project. I am going to keep them there to test the navigation in the deployed application hosted in WinUI:

1. Start by adding a **Tasks** component to the project. Do this by entering `dotnet new razorcomponent -n Tasks -o Pages` in the VS Code **TERMINAL** window. This will create a **Tasks** Razor component in the **Pages** folder.
2. Double-click **Tasks.razor** in the **Pages** folder in the **Explorer** window to open it in the editor. As an alternative to the Explorer window, the C# Dev Kit extension should have added a **Solution Explorer** view to the bottom of the left pane. Use this view if you like. The file contains the following code:

```
<h3>Tasks</h3>
@code {
}
```

Razor files contain a combination of HTML markup and C# code, with the HTML at the top of the file, and the C# inside the **@code**

block at the bottom of the file. We'll see how these two sections can interact as we move along.

3. Add `@page "/tasks"` as the first line of the `Tasks.razor` file. This will allow the application to route to the page using `/tasks` on the URL.
4. Before we add the page contents, let's add the new navigation item for it. Open `NavMenu.razor` from the **Shared** folder in **Explorer**.
5. Inside the `<nav>` element, add a new `<div>` before the closing `</nav>` tag:

```
<div class="@NavMenuCssClass nav-scrollable"
    @onclick="ToggleNavMenu">
    <nav class="flex-column">
    ...
        <div class="nav-item px-3">
            <NavLink class="nav-link" href="tasks">
                <span class="oi oi-list-rich" aria-
                    hidden="true"></span> Tasks
            </NavLink>
        </div>
    </nav>
</div>
```

6. Run the application with `dotnet run` to make sure the new menu option appears and you can navigate to the new page with the **Tasks** header:

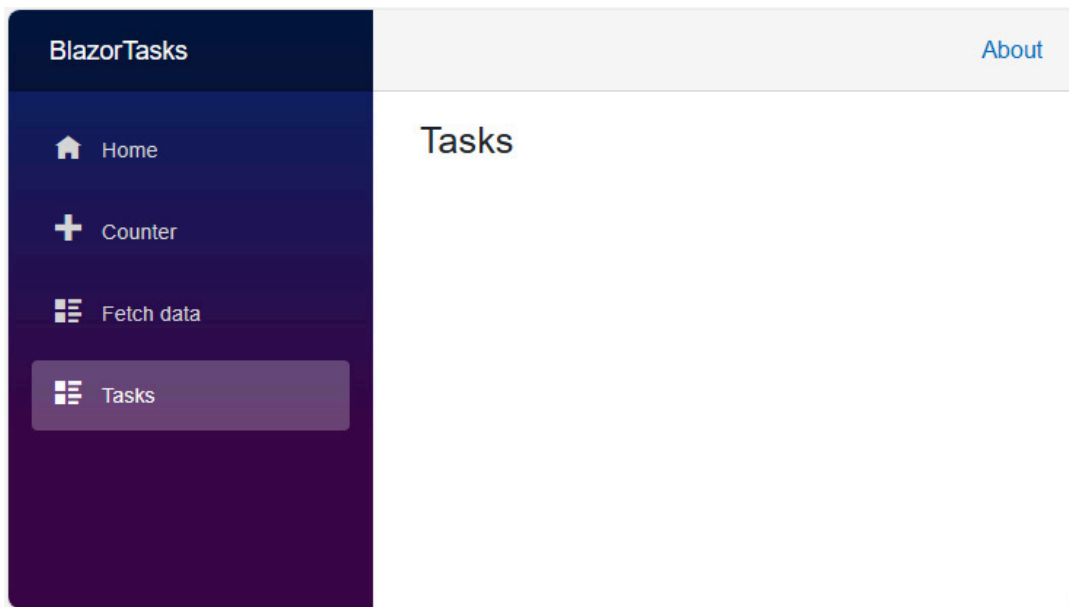


Figure 12.8 – Navigating to the new Tasks page

- Next, use **File | New File** at the root of the project and name the file **TaskItem.cs**. This will be the model class for tasks. Add the following code to the new file:

```
namespace BlazorTasks
{
    public class TaskItem
    {
        public string? Name { get; set; }
        public bool IsComplete { get; set; }
    }
}
```

- Open **Tasks.razor** and add the following code to create an unordered list of tasks by iterating over a list of tasks contained in the **@code** block:

```
@page "/tasks"
<h3>Tasks</h3>
<ul>
    @foreach (var task in taskList)
    {
        <li>@task.Name</li>
    }
}
```

```

</ul>
<input placeholder="Enter new task..." />
<button>Add task</button>
@code {
    private IList<TaskItem> taskList = new
        List<TaskItem>();
}

```

Notice how Razor files allow you to blend C# code and HTML markup. We have a C# **foreach** within ****, and inside **foreach**, we're adding **** elements that again contain C# code to get each **task.Name**. This is powerful stuff. We've also added an **input** field to enter a new task and a **button** to add the task. We'll add some code to make **button** functional next.

9. Add a **newTask** private variable and a new method to the **@code** block named **AddTask**. This method will add a new task to the **taskList** collection:

```

private string newTask;
private void AddTask()
{
    if (!string.IsNullOrEmpty(newTask))
    {
        taskList.Add(new TaskItem { Name = newTask });
        newTask = string.Empty;
    }
}

```

10. Finally, add some data binding code to the **input** and **button** elements on the page. The **input** will bind to the **newTask** variable, and the **onclick** event of **button** will trigger the **AddTask** method to run:

```

<input placeholder="Enter new task..." @bind="newTask"
/>
<button @onclick="AddTask">Add task</button>

```

11. Now, run the application and test the controls. You should be able to add some tasks to the list:

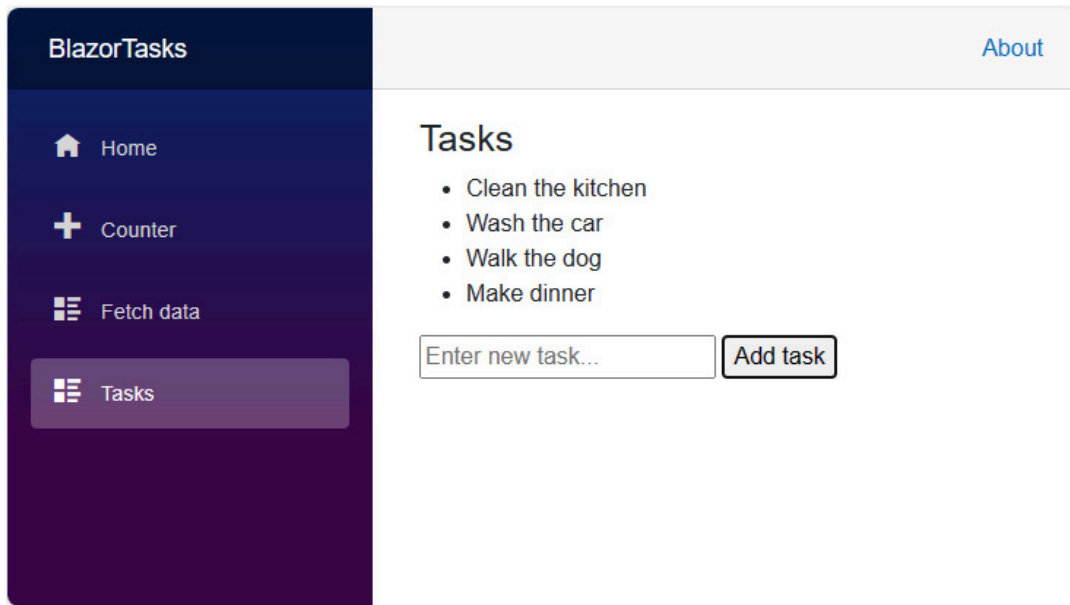


Figure 12.9 – Adding some tasks to the task list in BlazorTasks

This works great, but now that we have some tasks to do, we don't have any way to mark them as done. Let's take care of that next.

12. The first step is to make each list item **checkbox** that users can check when they complete a task. We are also binding **task.Name** to an **input** field so users can edit the name of each task:

```
<ul>
  @foreach (var task in taskList)
  {
    <li>
      <input type="checkbox"
        @bind="task.IsComplete" />
      <input @bind="task.Name" />
    </li>
  }
</ul>
```

13. Next, in case the list gets lengthy, let's use some data binding to display the number of incomplete tasks as part of the page header:

```
<h3>Tasks - (@taskList.Count(task =>
  !task.IsComplete)) Incomplete</h3>
```

14. Run the application again, and start working on your task list:

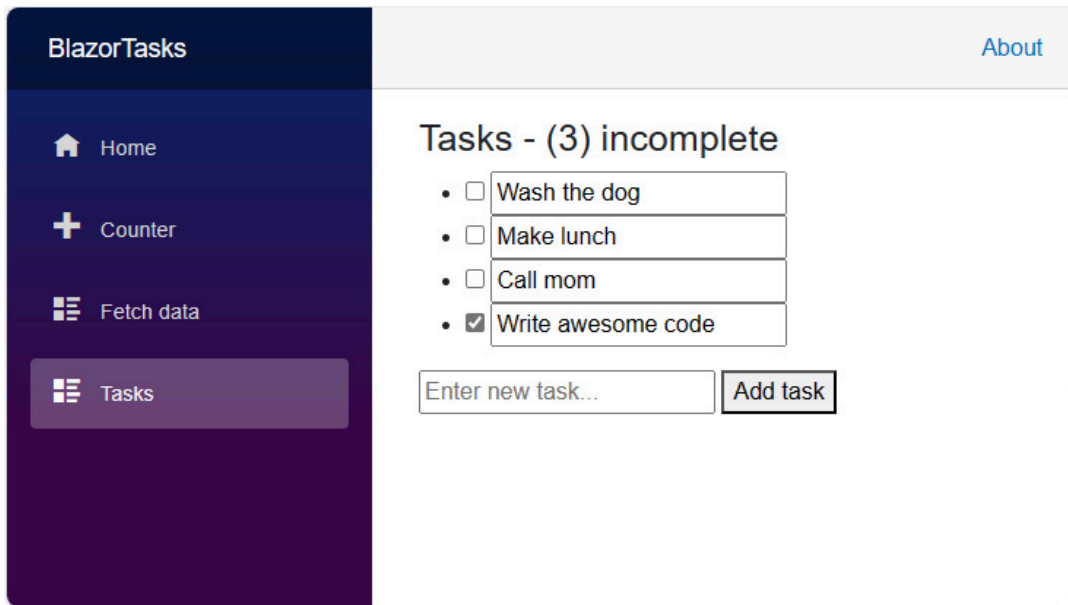


Figure 12.10 – Adding and completing tasks in the BlazorTasks application

You may have noticed that the tasks do not save between sessions. The `taskList` is an in-memory collection for now. To persist it between sessions, you would need to add service calls to save the data in a server-side data store. Creating this service is beyond the scope of this chapter, and I will leave it as an exercise for you.

NOTE

All of these steps can also be taken with Visual Studio or Visual Studio for Mac if you prefer using a full-featured IDE. This Microsoft Learn documentation page has information on how to debug a Blazor Wasm app in both of these tools, as well as VS Code:

<https://learn.microsoft.com/aspnet/core/blazor/debug>.

Now that we have a functioning task-tracking web client, we can move on to the next step. It's time to deploy our Blazor app to the cloud.

Exploring Blazor Wasm deployment options

Running and debugging the Blazor project locally is great while we're developing the solution, but when it's time to share your application with the world, we will need to host it in the cloud. There are many cloud hosting options for typical ASP.NET Core applications, and Blazor Wasm applications have even more. Sites that run entirely on the client can be hosted as static files on the server, meaning that the server simply serves up the files when it receives a request. There is no server-side execution required.

Let's start by reviewing some of the available hosting options for Blazor WebAssembly deployments.

Deployment options for Blazor Wasm projects

There are several hosting options for our Blazor project. We are going to discuss a few of the most popular solutions today: **GitHub Pages**, **Azure App Service**, **Azure Static Web Apps**, and two options on **Amazon Web Services (AWS)**. For an in-depth exploration of options either hosted with ASP.NET Core or as static files, Microsoft Learn has a great article: <https://learn.microsoft.com/aspnet/core/blazor/host-and-deploy/webassembly>.

Amazon Web Services

With AWS, a Blazor Wasm site can be hosted with ASP.NET Core in **Elastic Container Service (ECS)** (<https://aws.amazon.com/ecs/>) and **Fargate**. The ECS solution uses **Docker** to create the container to be hosted in the cloud. The site is then served through Fargate (<https://aws.amazon.com/fargate/>), the AWS compute engine for containers. To read more about this solution for ASP.NET Core projects,

the AWS blog has a great article detailing the steps:

<https://aws.amazon.com/blogs/compute/hosting-asp-net-core-applications-in-amazon-ecs-using-aws-fargate/>.

For a static hosting option with AWS, static pages can be hosted using **Amazon S3** storage (<https://aws.amazon.com/s3/>) and **CloudFront** (<https://aws.amazon.com/cloudfront/>). An Amazon S3 bucket is a cloud file storage solution. Your `wwwroot` folder will be copied to S3 storage, and CloudFront handles serving the static files from the S3 bucket. This article details how to create and deploy a Blazor Wasm application in AWS: <https://aws.amazon.com/blogs/developer/run-blazor-based-net-web-applications-on-aws-serverless/>.

Now, let's see how to serve static files through **GitHub**.

GitHub Pages

GitHub Pages (<https://pages.github.com/>) are static websites served directly from GitHub repositories. You can maintain your site on GitHub and configure **GitHub Actions** to deploy the site to GitHub Pages. Microsoft MVP Niels Swinburghe has a step-by-step guide for deploying Blazor Wasm projects to GitHub Pages on his personal blog: <https://swimbunger.net/blog/dotnet/how-to-deploy-aspnet-blazor-webassembly-to-github-pages>. GitHub Pages are free, but *standard* user accounts can only host pages from the *default* GitHub branch.

In the next section, we will be using GitHub Actions with our project to deploy to Azure. But now, let's review two of the available Azure hosting solutions.

Azure App Service

Azure App Service (<https://azure.microsoft.com/products/app-service/>) is a great option to use if you want your Blazor app hosted on an ASP.NET Core web server. There are Windows and Linux servers available with App Service, but currently, only Windows instances are

supported with Blazor WebAssembly apps. Microsoft Learn has extensive documentation on deploying ASP.NET Core applications to App Service: <https://learn.microsoft.com/aspnet/core/host-and-deploy/azure-apps/>.

Now, let's look at another Azure option. This one is specifically for deploying static sites such as Blazor Wasm.

Azure Static Web Apps

Azure Static Web Apps (<https://azure.microsoft.com/products/app-service/static/>) is a service to host and serve static web applications such as Blazor Wasm. It offers easy deployment through GitHub Actions, free SSL certificates, custom domains, and easy integration with **Azure Functions**.

For full documentation on Static Web Apps, including information about using it with other SPA websites, Microsoft Learn has documentation, guides, and **Training** (<https://learn.microsoft.com/training/>) content available at <https://learn.microsoft.com/azure/static-web-apps/>.

We are going to use Static Web Apps to host our Blazor application. Let's do that now!

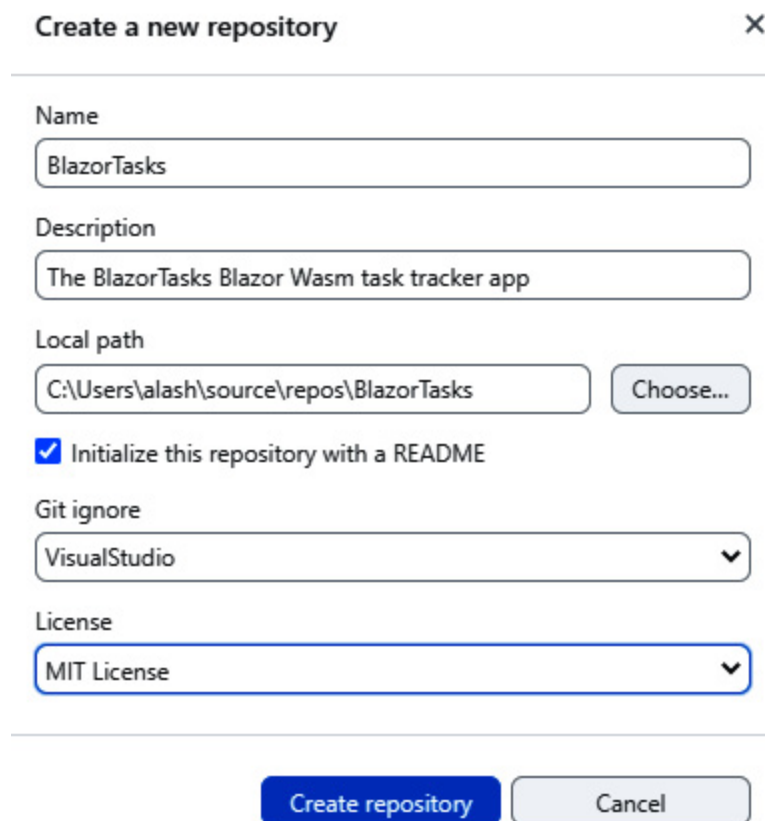
Publishing Blazor to Azure Static Web Apps hosting

In this section, we are going to host our **BlazorTasks** application in the cloud by pushing the source to GitHub, creating an Azure Static Web Apps app, and configuring GitHub Actions to publish the app to Azure with every commit to the main branch. Let's start by pushing our code to GitHub.

Pushing the project to GitHub

To push your code to a GitHub repository, you can either use the Git CLI (<https://git-scm.com/downloads>) or the **GitHub Desktop** (<https://desktop.github.com/>) application. We will use GitHub Desktop in this example:

1. Download and install GitHub Desktop. When the installation is complete, launch the application.
2. If your local project is not part of a Git repository yet, choose **File** | **New Repository**. If you already have a local repository for your project, you can skip to the next step:



The screenshot shows the 'Create a new repository' window in GitHub Desktop. It has a title bar with a close button (X). The window contains several input fields and checkboxes:

- Name:** A text box containing 'BlazorTasks'.
- Description:** A text box containing 'The BlazorTasks Blazor Wasm task tracker app'.
- Local path:** A text box containing 'C:\Users\alash\source\repos\BlazorTasks' and a 'Choose...' button.
- Initialize this repository with a README:** A checked checkbox.
- Git ignore:** A dropdown menu showing 'VisualStudio'.
- License:** A dropdown menu showing 'MIT License'.

At the bottom, there are two buttons: 'Create repository' (highlighted in blue) and 'Cancel'.

Figure 12.11 – Creating a new local GitHub repository for the BlazorTasks application

Name the repository something like **BlazorTasksWasm** or **BlazorTasks**, optionally add a description, and browse to **Local path** for your project. It's a good practice to have a README, a **Git ignore** file, and

License. So, choose each of these options. When you're done, click **Create repository**. After completing this step, move ahead to *step 4*.

3. If you created your Blazor project in a local Git repository, you can select **File | Add Local Repository**. Browse to the folder where you have the **BlazorTasks** project and select it. If you do not have a Git repository there yet, the application will prompt you to create one:

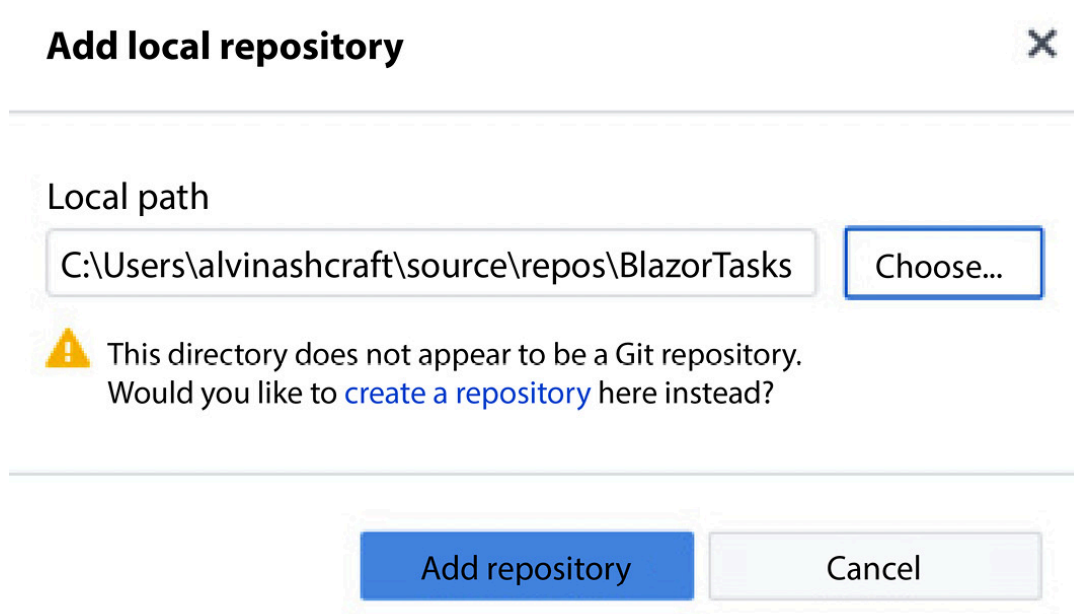


Figure 12.12 – Adding a local repository

4. In this step, we'll publish the local repository to GitHub. If you don't have a GitHub account, you can create one at <https://github.com/>. When you're ready to go, make sure your **BlazorTasksWasm** repository is selected for **Current repository** and click the **Publish repository** button:

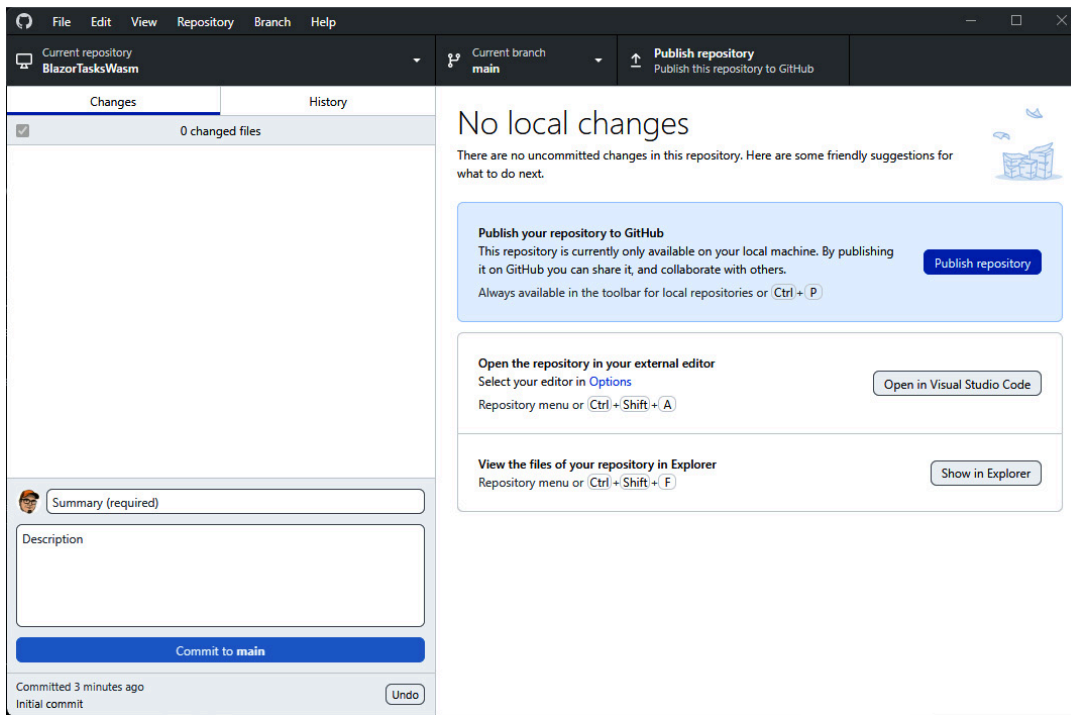


Figure 12.13 – Publishing the local repository to GitHub

If you have the **Keep this code private** option on the dialog that appears, you can uncheck it.

5. View the repository on GitHub to make sure it has been published correctly:

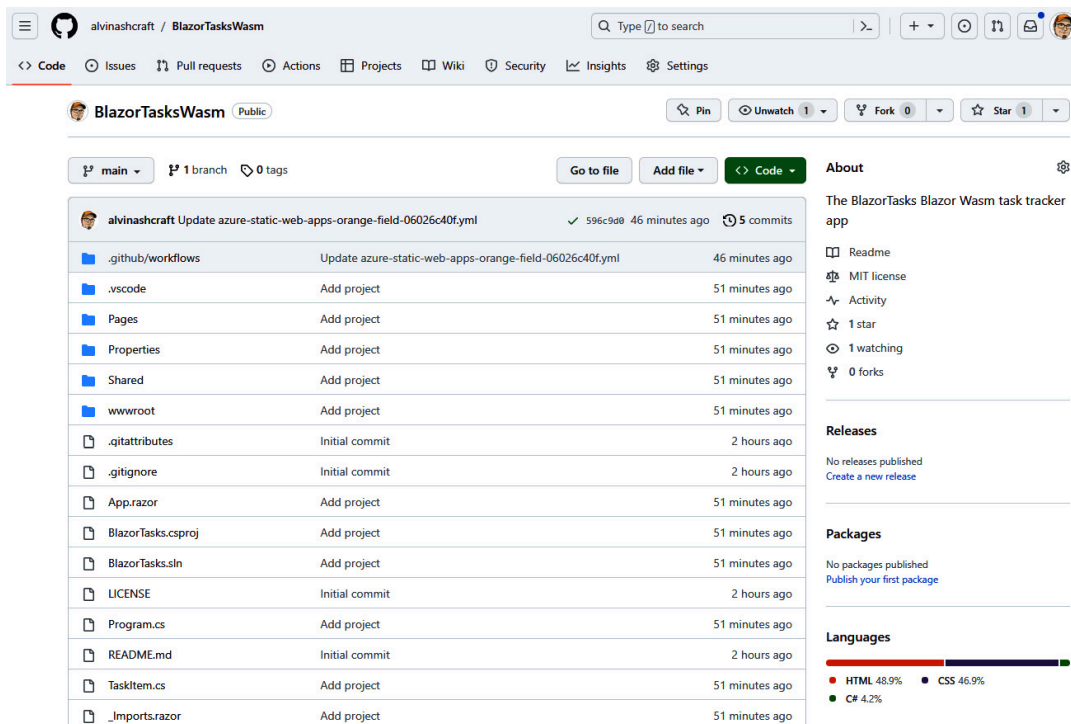


Figure 12.14 – The BlazorTasks code on GitHub

Now, the code is ready to be published to Azure. Let's do that next.

Creating an Azure Static Web Apps resource

Let's walk through creating a new Azure Static Web Apps app:

1. To start, if you don't have an Azure account yet, you can create a free trial account at <https://azure.microsoft.com/>. The site will walk you through the steps to create a new account.
2. Log in to the Microsoft account associated with your Azure account at <https://portal.azure.com/>.
3. From the portal home page, click **Create a resource** at the top under **Azure services**.
4. On the **New** page, search for **static** and select **Static Web App**:

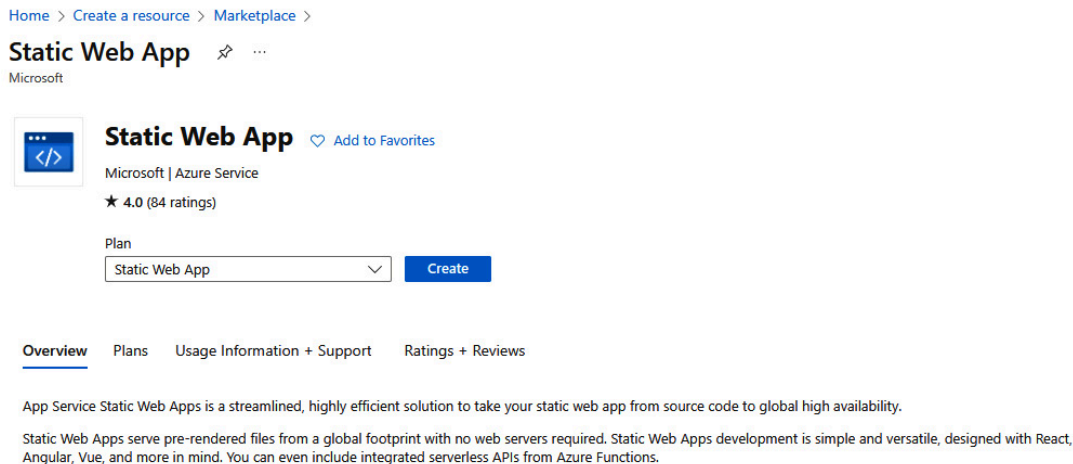


Figure 12.15 – Creating a new Static Web App

5. Click **Create**. On the **Create Static Web App** page, select your free trial subscription. You will also need to select a resource group. Resource groups are a way to group a related set of Azure resources. Create a new **Resource Group** and name it **BlazorTasksWasm**. Give the resource a name, select a **Region** option that makes sense for you or your users, and select the **Free** option for **SKU**. Choose **GitHub** for **Deployment details**. We'll link to GitHub in the next step:

[Home](#) > [Create a resource](#) > [Marketplace](#) > [Static Web App](#) >

Create Static Web App ...

[Basics](#) [Tags](#) [Review + create](#)

App Service Static Web Apps is a streamlined, highly efficient solution to take your static app from source code to global high availability. Pre-rendered content is distributed globally with no web servers required. [Learn more](#) 

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * 	Visual Studio Enterprise Subscription ▼
Resource Group * 	(New) BlazorTasksWasm ▼

[Create new](#)

Static Web App details

Name *	BlazorTasksWasm ✓
--------	---

Hosting plan

The hosting plan dictates your bandwidth, custom domain, storage, and other available features. [Compare plans](#)

Plan type	☒ Free: For hobby or personal projects
	☐ Standard: For general purpose production apps

Azure Functions and staging details

Region for Azure Functions API and staging environments *	East US 2 ▼
---	-----------------------------------

Deployment details

Source	☒ GitHub ☐ Azure DevOps ☐ Other
--------	--

GitHub account	Sign in with GitHub
----------------	--------------------------------

Figure 12.16 – Configuring the new Static Web Apps resource

- Next, click the **Sign in with GitHub** button to link your GitHub account. Linking the accounts is necessary for the GitHub files to be deployed to the Azure site. After it has linked your accounts, select the **Organization**, **Repository**, and **Branch** names for your **BlazorTasksWasm** repository:

GitHub account: alvinashcraft [Change account](#) ⓘ

Information: If you can't find an organization or repository, you might need to enable additional permissions on GitHub. You must have write access to your chosen repository to deploy with GitHub Actions.

Organization *

Repository *

Branch *

Figure 12.17 – Entering GitHub details for the Static Web Apps resource

7. In the **Build Details** section, select **Blazor** from the **Build Presents** dropdown. **App location** will be `.` (*just a dot*), **Api location** can be left blank, and **Output location** will be `wwwroot`.
8. Click **Review + create**. Review the **Summary** page to make sure everything looks correct and click **Create**. Azure will take a few minutes to create the new resource. When it's done, you can click **Go to resource**.

The Static Web Apps resource is ready to go. Azure created our GitHub Actions deployment for us. Let's review what it did and then review the website.

Publishing an application with GitHub Actions

Typically, this is where we would configure GitHub Actions to build our project in the GitHub repository and publish it to the Azure resource. However, the Azure Static Web App configuration took care of that step for us. Let's review what it did:

1. Navigate to your project on GitHub and click the **Actions** tab. You will see that Azure has created a workflow named **Azure Static Web Apps CI/CD**:

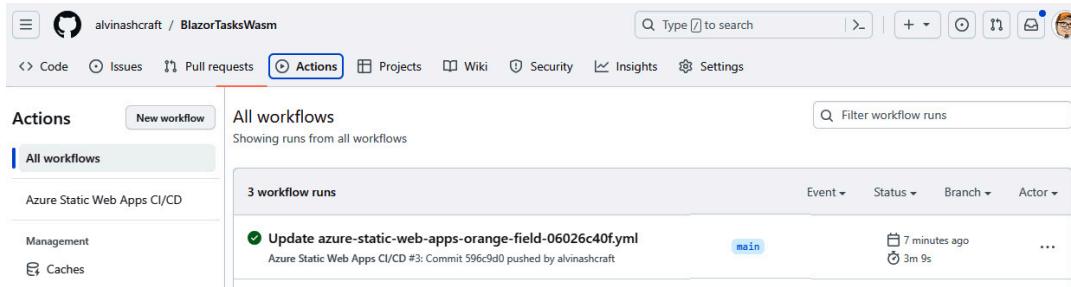


Figure 12.18 – Viewing the workflows for BlazorTasks

2. Select the **Azure Static Web Apps CI/CD** workflow and click the **.yaml** file hyperlink that appears just under the **Azure Static Web Apps CI/CD** title. The **.yaml** file will appear in an editor.
3. Review the file contents. You'll see a section for **build_and_deploy_job**. This step will take the latest committed code, build it, and deploy it to the app service that we configured in Azure.
4. You can verify that the site has been published to Azure by navigating to the **BlazorTasksWasm** resource in the Azure portal and clicking on the URL for the site:

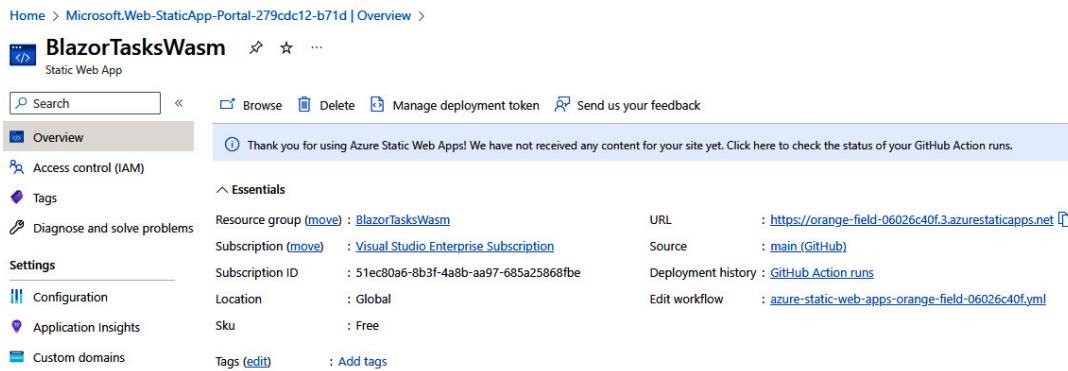


Figure 12.19 – The BlazorTasks resource home page in the Azure portal

5. The Blazor site will open in a new tab in your browser. Click the **Tasks** item in the navigation menu and verify that the application works as expected:

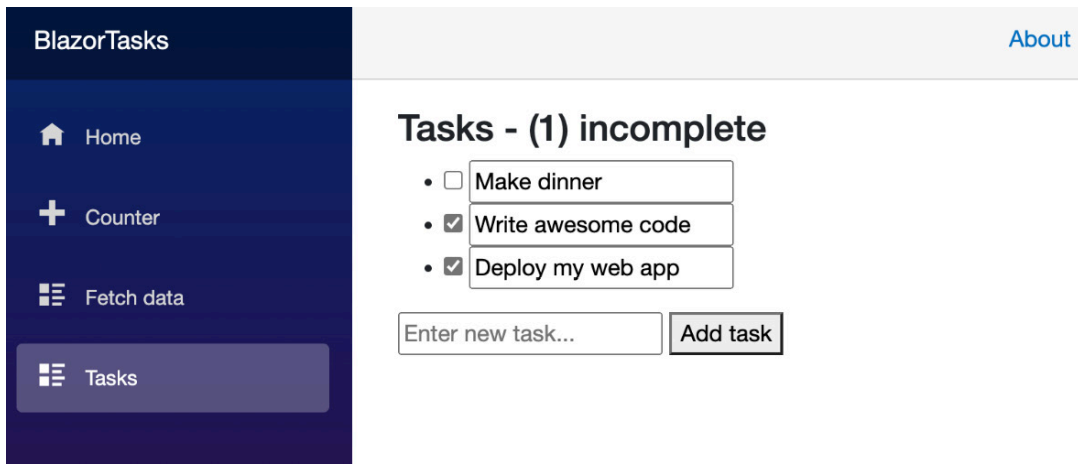


Figure 12.20 – Running BlazorTasks in the static cloud

We now have a public-facing static website running the Blazor Wasm application. Now, we're ready to run the web application inside a WinUI project.

Hosting your Blazor application in the WinUI WebView2

We're on the home stretch. We created a Blazor Wasm application, pushed the source code to GitHub, and Azure configured GitHub Actions to publish the application to Azure Static Web Apps with every commit. The last step is to create a simple WinUI 3 project and add a **WebView2** control to **MainWindow**:

1. You can start by either creating a new **Blank App, Packaged (WinUI in Desktop)** named **BlazorTasksHost** in Visual Studio or opening the starter project from GitHub:
<https://github.com/PacktPublishing/-Learn-WinUI-3/tree/master/Chapter12/Start/BlazorTasksHost>.
2. Open **MainWindow.xaml** and update the window to host **Grid** that contains the **WebView2** control. Set the **Source** property to the URL of your **BlazorTasksWasm** site:

```
<Grid>
```

```
<WebView2 Source="https://you-custom-url-  
0af06780d.azurestaticapps.net/" />  
</Grid>
```

3. Remove the unused button click event handler in **MainWindow.xaml.cs** to prevent compilation errors.
4. Run the application, and you'll see the **BlazorTasksWasm** application load as if it were a Windows application:

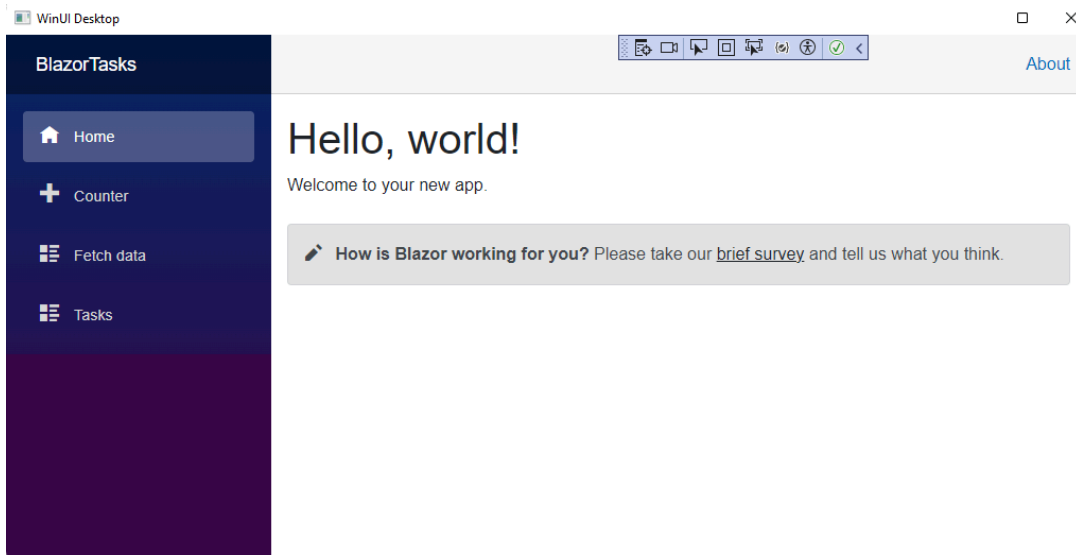


Figure 12.21 – Running BlazorTasksWasm in a WinUI application

You can test the app in the web view. Because it's all client-side code, you can even disconnect from your network and keep using the app. The **Tasks** page will continue to function offline.

Now, any updates you make to the Blazor application will be immediately pushed to all your users when you commit to GitHub. This is a compelling way for web developers to reach more Windows users.

NOTE

*If you would like to explore Blazor and WinUI integration further, you can check out this blog post by Thomas Claudius Huber. In the post, he experiments with calling a method in the Blazor app from the WinUI host application by executing a script through the **WebView2** control:*

<https://www.thomasclaudiushuber.com/2020/02/18/hosting-blazor-app-in-winui-3-with-webview-2-and-call-blazor-component-method-from-winui/>.

Let's wrap up with a summary of what we've covered in this chapter.

Summary

In this chapter, we learned about ASP.NET Core Blazor. You created a simple task-tracking application with Blazor Wasm and published it to Azure Static Web Apps with GitHub Actions. From here, you could use ASP.NET Core Identity to integrate an application login and save the task data to **Azure SQL**, **Azure Cosmos DB**, or another cloud-based data store. This would allow personalizing the task list for each user and saving its state. We created a WinUI 3 application to run the Blazor client on Windows, but you could also send users directly to your site or create a JavaScript-based PWA for desktop and mobile clients. For more information about creating a PWA with Blazor WASM, check out this Microsoft blog post:

<https://devblogs.microsoft.com/visualstudio/building-a-progressive-web-app-with-blazor/>.

NOTE

*To learn more about building web applications with Blazor, you can read *Web Development with Blazor* by Jimmy Engstrom. Here's the Amazon link: <https://www.amazon.com/dp/1803241497/>*

In the next chapter, ***Chapter 13***, *Take Your App Cross-Platform with Uno Platform*, we will explore what **Uno Platform** can do for WinUI developers.

Questions

1. What is the name of the Blazor hosting model in .NET 7 and earlier that runs all application logic in the browser?
2. Which Blazor hosting model is less scalable?
3. What is the name of the syntax used in Blazor UI files?
4. Which .NET CLI command will compile and run the project in the current folder?
5. What is the name of GitHub's product that hosts static websites?
6. Which Azure product hosts static websites?
7. What WinUI 3 control can load web content in a Chromium-based browser control?
8. What is the name of GitHub's **Continuous Integration/Continuous Delivery (CI/CD)** solution?